

CLAUDE.md: DigitalFlyer Growth
Production build specification. Self-contained, hand directly to Claude Code. Sub-brand of DigitalFlyer SA.

0. Before you build anything

This project integrates with Meta's Graph API, Pixel, and Conversions API. Meta changes these frequently and without much notice (attribution windows removed January 2026, Offline Conversions API retired in favor of CAPI/Datasets in May 2025, a new one-click CAPI option shipped April 2026). Verify current CAPI endpoint versions, required fields, and event deduplication rules against Meta's live developer documentation before implementing Section 7, even though the pattern below is correct as of this writing.

1. What this is

DigitalFlyer Growth is a marketing execution platform, sold two ways:

1. As a premium software-with-a-service (SaaS) product, sub-branded under DigitalFlyer SA, that a South African business pays a monthly plan for.
2. As the internal engine DigitalFlyer SA's own agency team uses to deliver the marketing layer for existing retainer clients.

Both usages are the same application. A "growth client" record doesn't care whether it arrived through the agency's own retainer or a directly sold plan.

Core capabilities: conversion-optimized landing pages built from client data (not hand-coded per client), programmatic social media asset generation from testimonials and marketing copy, and a server-side Meta Conversions API pipeline so ad spend for budget-sensitive clients isn't wasted on bad attribution.

2. Onboarding model (self-serve, matches the finalized business plan)

No admin action is required to activate a new client. A public pricing page lists the three tiers (Foundation, Growth Engine, Enterprise), the business picks one, enters their name and email, and is sent to a Paystack-hosted checkout for that tier's subscription plan. Once Paystack confirms the subscription, the client is provisioned automatically and lands in the guided intake wizard (Section 6). You, as admin, see every new signup but don't need to do anything to make one active.

This means you can test the entire platform yourself by going through this exact flow as a real client, using DigitalFlyer SA as the business. That's the intended way to validate the build before the actual pilot campaign runs, use Paystack's test mode for a dry run first, then a real transaction when you're ready to treat DigitalFlyer SA as the first live tenant.

2.1 Provisioning flow

1. Client submits the pricing page form (business name, email, selected tier)
2. Server Action creates a Paystack subscription via the Subscriptions API and redirects to the hosted checkout
3. Paystack calls your webhook on successful payment

4. The webhook handler creates the `growth\_clients` row (status moves from nothing to `pending\_intake`), creates a Supabase Auth user for the email via a magic link, and the client's next login drops them straight into the intake wizard

```
``typescript
// app/api/webhooks/paystack/route.ts
import { NextResponse } from 'next/server'
import crypto from 'crypto'
import { createAdminClient } from '@lib/supabase/admin'

export async function POST(request: Request) {
  const rawBody = await request.text()
  const signature = request.headers.get('x-paystack-signature')

  const expected = crypto
    .createHmac('sha512', process.env.PAYSTACK_SECRET_KEY!)
    .update(rawBody)
    .digest('hex')

  if (signature !== expected) {
    return NextResponse.json({ error: 'Invalid signature' }, { status:
401 })
  }

  const event = JSON.parse(rawBody)

  if (event.event === 'subscription.create' || event.event ===
'charge.success') {
    const { customer, plan, subscription_code } = event.data
    const supabase = createAdminClient()

    // metadata carries business_name and selected tier, set when the
subscription was created
    const { business_name, tier } = event.data.metadata ?? {}

    await supabase.from('growth_clients').insert({
      business_name: business_name ?? customer.email,
      slug: slugify(business_name ?? customer.email),
      plan: tier ?? 'foundation', // 'foundation' | 'growth_engine' |
'enterprise'
      paystack_subscription_code: subscription_code,
      status: 'pending_intake',
    })

    // Send a Supabase Auth magic link to customer.email so they can log
in
    // and land in the intake wizard on next visit, implement via
    // supabase.auth.admin.generateLink({ type: 'magiclink', email:
customer.email })
  }

  return NextResponse.json({ received: true })
}
...

```

WhatsApp and email sending, already built for your other products, remain useful here as a **supplementary** channel, an abandoned-checkout

reminder, or a nudge to finish the intake wizard, but they are no longer the mechanism that creates the account. That happens automatically off the Paystack webhook.

2.2 A practical note on pricing before you can actually launch this

The business plan intentionally leaves pricing directional, pending real-world case studies. Paystack still needs an actual amount to charge, though, so before Sprint 1 is testable end to end, pick a concrete starting number per tier, the low end of each directional range is a sensible default (roughly R550 for Foundation, R1,400 for Growth Engine, R3,500 for Enterprise). These are launch prices for the case-study phase, not final pricing, and can change without a rebuild since the plan amount lives in Paystack's Plan configuration, not hardcoded in the app.

3. Roles

Role	Scope
`dfsa_admin`	You. Full access across all growth clients. Creates clients, views all data, can impersonate a client dashboard for support.
`growth_owner`	The one paying member per growth client in v1. Manages brand kit, testimonials, landing page content, and ad tracking connection.

Multi-seat teams per client (additional members beyond the owner) are explicitly out of scope for v1, see Section 10.

4. Tech stack

Same standing stack as the rest of DigitalFlyer SA, no new vendors:

- Next.js App Router, TypeScript, Vercel Pro (new project: `digitalflyer-growth`, same Vercel team, no added seat cost)
- Supabase (Cape Town), either a new project or a new schema inside `dfsa-prod`/`dfsa-staging`, see Section 5.1 for the tradeoff
- Tailwind CSS, Shadcn/ui
- Paystack Subscriptions API for billing
- Meta Graph API (Pixel/CAPI) direct, no third-party tag manager
- `@vercel/og` (built into Next.js via `ImageResponse`) for programmatic image generation. Bannerbear is explicitly rejected: it is a new paid vendor solving a problem `@vercel/og` already solves inside the stack you've already paid for.
- Cloudflare for DNS/SSL on client-facing landing page domains
- Resend or Supabase's built-in email (confirm current option) for invite emails, evaluate at build time since this is the one piece not yet decided in your existing stack

4.1 Domain and routing strategy

DigitalFlyer Growth is a fully separate project with its own domain, not a subdomain or subfolder of the existing DigitalFlyer.co.za platform. That platform continues to run independently; Growth shares the brand name but not the domain, codebase, or infrastructure. Domain name itself is TBD, register it as its own asset (a `.co.za` domain matching the sub-brand name is the obvious default, e.g. `digitalflyergrowth.co.za`) and

point it at the `digitalflyer-growth` Vercel project through Cloudflare, same pattern as any other client domain in your stack.

If DigitalFlyer.co.za and DigitalFlyer Growth are ever merged into one platform later, that becomes a domain migration exercise at that point (redirects, DNS changes), not something to design around now.

5. Data model

5.1 New Supabase project vs new schema

Recommendation: a ****new Supabase project**** (`dfsa-growth-prod` / `dfsa-growth-staging`), not a schema inside the existing directory database. Reasoning: this data (ad tracking tokens, CAPI event logs, Paystack subscription state) is more sensitive and higher-liability than directory listings, and keeping it physically separate limits blast radius if either project is ever compromised. Cost impact is roughly \$10/month in additional compute per your budget document, acceptable given what it protects.

5.2 Schema

```
```sql
create table growth_clients (
 id uuid primary key default gen_random_uuid(),
 business_name text not null,
 slug text unique not null,
 plan text not null, -- 'foundation' | 'growth_engine' | 'enterprise'
 paystack_subscription_code text,
 status text not null default 'pending_intake', -- 'pending_intake' |
'active' | 'paused' | 'cancelled'
 brand_primary_color text,
 brand_secondary_color text,
 logo_path text,
 meta_pixel_id text, -- null for 'foundation' tier clients, they never
connect Meta ads
 meta_ad_account_id text,
 created_at timestamptz default now()
);

-- Access tokens are sensitive. Store encrypted at the application layer
before insert,
-- never store a raw long-lived Meta access token in plaintext in this
column.
create table growth_client_secrets (
 growth_client_id uuid primary key references growth_clients(id) on
delete cascade,
 meta_capi_access_token_encrypted text,
 updated_at timestamptz default now()
);

create table growth_members (
 id uuid primary key default gen_random_uuid(),
 user_id uuid references auth.users(id) not null,
 growth_client_id uuid references growth_clients(id) not null,
 role text not null default 'growth_owner',
 unique (user_id, growth_client_id)
);
```

```

create table testimonials (
 id uuid primary key default gen_random_uuid(),
 growth_client_id uuid references growth_clients(id) not null,
 author_name text not null,
 quote text not null,
 rating int check (rating between 1 and 5),
 photo_path text,
 created_at timestamptz default now()
);

create table generated_assets (
 id uuid primary key default gen_random_uuid(),
 growth_client_id uuid references growth_clients(id) not null,
 testimonial_id uuid references testimonials(id),
 template text not null, -- 'testimonial-square' | 'quote-story' |
 'promo-banner'
 image_path text not null,
 created_at timestamptz default now()
);

create table landing_pages (
 id uuid primary key default gen_random_uuid(),
 growth_client_id uuid references growth_clients(id) not null,
 slug text not null,
 headline text not null,
 subheadline text,
 cta_label text not null default 'Get Started',
 cta_href text not null,
 published boolean default false,
 unique (growth_client_id, slug)
);

create table capi_events (
 id uuid primary key default gen_random_uuid(),
 growth_client_id uuid references growth_clients(id) not null,
 event_name text not null, -- 'Lead' | 'Purchase' |
 'CompleteRegistration'
 event_id text not null, -- used for Pixel/CAPI deduplication
 fbclid text,
 hashed_email text,
 hashed_phone text,
 payload jsonb,
 sent_at timestamptz default now(),
 response_status int
);

-- Captures a landing page conversion (e.g. a membership sign-up lead),
-- distinct from `testimonials`, which holds client-submitted marketing
content.
create table leads (
 id uuid primary key default gen_random_uuid(),
 growth_client_id uuid references growth_clients(id) not null,
 landing_page_id uuid references landing_pages(id),
 name text not null,
 email text not null,
 phone text,
 fbclid text,
 event_id text not null,

```

```

 created_at timestamptz default now()
);

alter table leads enable row level security;

create policy "growth members read own client leads"
on leads for select
using (
 exists (
 select 1 from growth_members
 where growth_members.growth_client_id = leads.growth_client_id
 and growth_members.user_id = auth.uid()
)
);

```

```

-- No public insert policy: leads are only written server-side via the
-- Server Action using the service role client, never directly from the
browser.
```

```

5.3 RLS policies (pattern, apply to every tenant table)

```

```sql
alter table growth_clients enable row level security;
alter table testimonials enable row level security;
alter table landing_pages enable row level security;
alter table generated_assets enable row level security;
alter table capi_events enable row level security;

-- Members read their own client's data
create policy "growth members read own client"
on testimonials for select
using (
 exists (
 select 1 from growth_members
 where growth_members.growth_client_id = testimonials.growth_client_id
 and growth_members.user_id = auth.uid()
)
);

-- Members write their own client's data
create policy "growth members write own client"
on testimonials for insert
with check (
 exists (
 select 1 from growth_members
 where growth_members.growth_client_id = testimonials.growth_client_id
 and growth_members.user_id = auth.uid()
)
);

-- capi_events and growth_client_secrets are never client-readable
directly,
-- only accessed server-side via the service role key inside Route
Handlers.
-- Do not write a client-facing select policy for these two tables.
```

```

6. Client intake wizard (multi-step, Zod, Server Actions)

Frictionless per your brief: every extra field costs conversions, so this wizard is capped at four short steps, auto-saves progress after each step, and takes under two minutes. Step 4 (Meta ad account connection) only appears for `growth\_engine` and `enterprise` tier clients, a `foundation` tier client finishes after step 3 and is never asked to connect anything they don't need.

```
```typescript
// lib/schemas/intake.ts
import { z } from 'zod'

export const step1Schema = z.object({
 businessName: z.string().min(2),
 contactEmail: z.string().email(),
})

export const step2Schema = z.object({
 brandPrimaryColor: z.string().regex(/^#[0-9A-Fa-f]{6}$/),
 brandSecondaryColor: z.string().regex(/^#[0-9A-Fa-f]{6}$/),
})

export const step3Schema = z.object({
 headline: z.string().min(5).max(80),
 subheadline: z.string().min(10).max(160),
 ctaLabel: z.string().min(2).max(30),
})

// Only shown when growth_clients.plan is 'growth_engine' or 'enterprise'
export const step4Schema = z.object({
 metaPixelId: z.string().optional(),
 metaAdAccountId: z.string().optional(),
})
```

```typescript
// app/onboard/actions.ts
'use server'
import { step1Schema } from '@lib/schemas/intake'
import { createServerClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'

export async function saveStep1(formData: FormData) {
 const parsed = step1Schema.safeParse({
 businessName: formData.get('businessName'),
 contactEmail: formData.get('contactEmail'),
 })
 if (!parsed.success) {
 return { error: parsed.error.flatten().fieldErrors }
 }

 // The client is already authenticated at this point, they arrived here
 // via the magic link Supabase Auth sent after Paystack confirmed
 payment
 // (Section 2.1), not via a token we have to validate ourselves.
 const supabase = createServerClient()
 const { data: { user } } = await supabase.auth.getUser()
```

```

 if (!user) {
 return { error: 'Your session has expired, please use the link from
your email again.' }
 }

 const admin = createAdminClient()
 const { data: membership } = await admin
 .from('growth_members')
 .select('growth_client_id')
 .eq('user_id', user.id)
 .single()

 if (!membership) {
 return { error: 'No account found for this login.' }
 }

 await admin
 .from('growth_clients')
 .update({ business_name: parsed.data.businessName })
 .eq('id', membership.growth_client_id)

 return { success: true, nextStep: 2 }
 }
 ...

```

Auto-save on every step submit (not on final submit only) is what gives you the "under 30 seconds per step, no data lost" behavior your brief asked for. Each step is its own Server Action, and the wizard component tracks current step in local state, reading initial values back from the database on mount so a client can close the tab and resume later simply by logging back in, no token or link expiry to worry about since access is tied to their authenticated account, not a one-time link.

## ## 7. High-conversion landing page requirements

Every generated landing page must satisfy three rules, enforced as component-level requirements Claude Code checks against, not just guidance:

1. **Five-second rule above the fold.** The hero section is a fixed layout: `` with logo, a one-line value proposition answering "what do you do," a sub-headline answering "how does it make my life better," and a single high-contrast CTA button answering "how do I buy or join." No hero section ships without all three elements present; this is a lint-level requirement for the landing page template component, not a suggestion.
2. **Trust elements adjacent to the CTA**, not buried in a footer: a testimonial slider fed from the `testimonials` table (same data reused in Section 8's asset generation, single source of truth), and a payment trust badge reading "Secure payment via Paystack" (not PayFast or Yoco, per your locked-in processor).
3. **Page speed.** Served from Vercel's edge network with Johannesburg/Cape Town PoPs in range, images optimized via `next/image`, no client-side rendering blocking the hero. Target under 1.5 seconds to largest contentful paint on a throttled mobile connection, verify with Lighthouse before marking any landing page sprint complete.



### 7.1 Dynamic per-client route (this is what serves the DigitalFlyer SA pilot, not a one-off page)

Every client, including DigitalFlyer SA as the first one, is served through the same route by slug, not a hardcoded page. This is the difference between building the platform once and building disposable pilot code.

```
```typescript
// app/g/[clientSlug]/page.tsx
import { createAdminClient } from '@lib/supabase/admin'
import { ConversionHero } from '@components/conversion-hero'
import { TrustBadges } from '@components/trust-badges'
import { LeadForm } from '@components/lead-form'
import { notFound } from 'next/navigation'

export default async function ClientLandingPage({
  params,
}: {
  params: { clientSlug: string }
}) {
  const supabase = createAdminClient()

  const { data: client } = await supabase
    .from('growth_clients')
    .select('*')
    .eq('slug', params.clientSlug)
    .eq('status', 'active')
    .single()

  if (!client) return notFound()

  const { data: landingPage } = await supabase
    .from('landing_pages')
    .select('*')
    .eq('growth_client_id', client.id)
    .eq('published', true)
    .single()

  if (!landingPage) return notFound()

  const { data: testimonials } = await supabase
    .from('testimonials')
    .select('*')
    .eq('growth_client_id', client.id)
    .limit(5)

  return (
    <main>
      <ConversionHero
        headline={landingPage.headline}
        subheadline={landingPage.subheadline}
        ctaLabel={landingPage.cta_label}
        primaryColor={client.brand_primary_color}
        logoPath={client.logo_path}
      />
      <TrustBadges testimonials={testimonials ?? []} />
    </main>
  )
}
```

```

        <LeadForm growthClientId={client.id} landingPageId={landingPage.id}
    />
    </main>
  )
}
...

```

DigitalFlyer SA's pilot page is simply a row in `growth\_clients` with `slug = 'digitalflyer-sa'` and one row in `landing\_pages`, no different from any future client. Nothing about this route is pilot-specific.

7.2 Lead capture Server Action

```

````typescript
// app/g/[clientSlug]/actions.ts
'use server'
import { z } from 'zod'
import { createAdminClient } from '@lib/supabase/admin'
import { sendCapiEvent } from '@lib/meta/capi'
import crypto from 'crypto'

const leadSchema = z.object({
 name: z.string().min(2),
 email: z.string().email(),
 phone: z.string().optional(),
})

export async function captureLead(
 growthClientId: string,
 landingPageId: string,
 fbclid: string | null,
 formData: FormData
) {
 const parsed = leadSchema.safeParse({
 name: formData.get('name'),
 email: formData.get('email'),
 phone: formData.get('phone'),
 })

 if (!parsed.success) {
 return { error: parsed.error.flatten().fieldErrors }
 }

 const eventId = crypto.randomUUID() // shared with any client-side
 Pixel call for dedup
 const supabase = createAdminClient()

 const { error: insertError } = await supabase.from('leads').insert({
 growth_client_id: growthClientId,
 landing_page_id: landingPageId,
 name: parsed.data.name,
 email: parsed.data.email,
 phone: parsed.data.phone,
 fbclid,
 event_id: eventId,
 })

 if (insertError) {

```

```

 return { error: 'Could not save your details, please try again.' }
 }

 // Direct function call, not a network hop to a separate API route.
 // Fire-and-forget: a slow or failed CAPI call should never block the
 // lead confirmation the visitor sees.
 sendCapiEvent({
 growthClientId,
 eventName: 'Lead',
 email: parsed.data.email,
 phone: parsed.data.phone,
 fbclid,
 eventId,
 }).catch(err => console.error('CAPI send failed', err))

 return { success: true }
}
...

```

## ## 8. Programmatic social asset generation

```

```typescript
// app/api/og/testimonial/[id]/route.tsx
import { ImageResponse } from 'next/og'
import { createAdminClient } from '@lib/supabase/admin'

export const runtime = 'edge'

export async function GET(
  request: Request,
  { params }: { params: { id: string } }
) {
  const supabase = createAdminClient()
  const { data: testimonial } = await supabase
    .from('testimonials')
    .select('*', growth_clients(business_name, brand_primary_color,
brand_secondary_color, logo_path))
    .eq('id', params.id)
    .single()

  if (!testimonial) {
    return new Response('Not found', { status: 404 })
  }

  const client = testimonial.growth_clients

  return new ImageResponse(
    (
      <div
        style={{
          height: '100%',
          width: '100%',
          display: 'flex',
          flexDirection: 'column',
          justifyContent: 'center',
          alignItems: 'center',
          backgroundColor: client.brand_primary_color,
          padding: '60px',

```

```

    }}
  >
  <div
    style={{
      fontSize: 42,
      color: 'white',
      textAlign: 'center',
      fontStyle: 'italic',
      maxWidth: '80%',
    }}
  >
    "{testimonial.quote}"
  </div>
  <div style={{ fontSize: 28, color: client.brand_secondary_color,
marginTop: 40 }}>
    {testimonial.author_name}
  </div>
</div>
),
{ width: 1080, height: 1080 }
)
}
...

```

Note the `runtime = 'edge'` requirement and the flexbox-only layout, `next/og` runs on Satori under the hood, which supports flexbox but not CSS grid or `position: fixed`. When a client submits a new testimonial, generate the image once and cache the result in `generated\_assets` with the storage path, rather than regenerating on every view, this keeps Vercel function invocations and edge compute cost down as you scale to 20 clients each generating assets weekly.

9. Meta Conversions API pipeline

****Verify current field requirements and endpoint version against Meta's live documentation before building this section.**** As of this writing, hashed PII (SHA-256, lowercased, trimmed) is required for email and phone matching, and an `event\_id` shared between any client-side Pixel call and the server-side CAPI call is required for deduplication.

This is a plain function, not a webhook. Meta never calls into this code, your own Server Actions call out to Meta, so it lives in `lib/meta/capi.ts` and gets imported directly wherever a conversion needs reporting (Section 7.2's `captureLead` action, for example), rather than being wrapped in its own API route. Avoid the extra network hop of calling your own route from your own server.

```

```typescript
// lib/meta/capi.ts
import crypto from 'crypto'
import { createAdminClient } from '@lib/supabase/admin'

function sha256(value: string) {
 return
 crypto.createHash('sha256').update(value.trim().toLowerCase()).digest('hex')
}

```

```

export async function sendCapiEvent({
 growthClientId,
 eventName,
 email,
 phone,
 fbclid,
 eventId,
}): {
 growthClientId: string
 eventName: 'Lead' | 'Purchase' | 'CompleteRegistration'
 email?: string
 phone?: string
 fbclid?: string | null
 eventId: string
}) {
 const supabase = createAdminClient()

 const { data: client } = await supabase
 .from('growth_clients')
 .select('meta_pixel_id')
 .eq('id', growthClientId)
 .single()

 const { data: secret } = await supabase
 .from('growth_client_secrets')
 .select('meta_capi_access_token_encrypted')
 .eq('growth_client_id', growthClientId)
 .single()

 if (!client?.meta_pixel_id ||
 !secret?.meta_capi_access_token_encrypted) {
 // Client is on a tier with no Meta connection (e.g. Foundation),
 this is expected, not an error.
 return { skipped: true }
 }

 const accessToken = decrypt(secret.meta_capi_access_token_encrypted) //
 implement decrypt() with your app's secret key

 const eventPayload = {
 data: [
 {
 event_name: eventName,
 event_time: Math.floor(Date.now() / 1000),
 event_id: eventId, // must match the client-side Pixel event_id
 action_source: 'website',
 user_data: {
 em: email ? [sha256(email)] : undefined,
 ph: phone ? [sha256(phone)] : undefined,
 fbc: fbclid ? `fb.1.${Date.now()}.${fbclid}` : undefined,
 },
 },
],
 },
],
}

const metaRes = await fetch(

```

```

`https://graph.facebook.com/v21.0/${client.meta_pixel_id}/events?access_token=${accessToken}`,
 {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(eventPayload),
 }
)

 await supabase.from('capi_events').insert({
 growth_client_id: growthClientId,
 event_name: eventName,
 event_id: eventId,
 fbclid,
 hashed_email: email ? sha256(email) : null,
 hashed_phone: phone ? sha256(phone) : null,
 payload: eventPayload,
 response_status: metaRes.status,
 })

 return { status: metaRes.status }
}
...

```

If you want a hit-able endpoint purely for manual testing (Postman, curl), add a thin wrapper at ``app/api/capi/test/route.ts`` that just calls ``sendCapiEvent()``, this is a debugging convenience, not part of the real call path, and is safe to leave out of v1 entirely.

Call this from the same Server Action that handles a landing page form submission (Section 7.2's pattern), server-side, right after the lead is saved. This means the event fires reliably regardless of ad blockers or iOS tracking restrictions on the visitor's device, which is the entire point of running CAPI at all, browser pixels alone are estimated to miss 30 to 50 percent of actual conversions industry-wide as of 2026.

Capture ``fbclid`` from the landing page URL query string on first visit and store it in a short-lived cookie, then read it back into the Server Action when the form is submitted, since it's required to link the eventual conversion back to the ad click.

## ## 10. Large file uploads (brand logos, testimonial photos)

Same signed-URL pattern as your core technical blueprint, applied to a ``growth-uploads`` bucket scoped by ``growth_client_id`` instead of ``client_id``. Do not duplicate a new upload mechanism, reuse the Route Handler and client-side pattern from that document directly.

## ## 11. Automated backups and monitoring

- Supabase Pro's automated daily backups cover ``dfsa-growth-prod`` the same as any other Pro project.
- Add a GitHub Action for a supplementary weekly ``pg_dump`` export to a private storage bucket, since this project holds Paystack subscription state and CAPI tokens that justify an extra layer beyond the default retention window.

- Add Better Stack/UptimeRobot monitors for each published landing page domain and for the `/api/capi/track` and `/api/og/testimonial/\*` endpoints specifically, a silently failing CAPI endpoint wastes client ad spend without any visible symptom on the frontend.

```
```yaml
# .github/workflows/growth-db-backup.yml
name: Weekly Growth DB Backup
on:
  schedule:
    - cron: '0 3 * * 0'
jobs:
  backup:
    runs-on: ubuntu-latest
    steps:
      - name: Dump database
        run: pg_dump ${ secrets.GROWTH_PROD_DB_URL } > backup.sql
      - name: Upload to storage
        run: |
          curl -X POST "${ secrets.SUPABASE_BACKUP_BUCKET_URL }" \
            -H "Authorization: Bearer ${ secrets.SUPABASE_SERVICE_ROLE_KEY }" \
            --data-binary @backup.sql
...
```
```

## ## 12. Build order (sprints)

### \*\*Sprint 0: Foundation\*\*

- New Vercel project `digitalflyer-growth`, new Supabase project pair (staging/prod)
- Core schema and RLS policies from Section 5
- Cloudflare DNS for the growth sub-brand domain
- Supabase Auth configured for magic-link sign-in (no separate admin auth needed, admin visibility is just a read-only view over the same data)

### \*\*Sprint 1: Public pricing page and self-serve billing\*\*

- `/pricing` page listing Foundation, Growth Engine, Enterprise with the launch prices from Section 2.2
- Paystack Subscriptions API integration and hosted checkout redirect
- `app/api/webhooks/paystack/route.ts` per Section 2.1, provisions the client and triggers the magic-link email on successful payment

### \*\*Sprint 2: Client intake wizard\*\*

- Wizard per Section 6, with step 4 conditional on tier
- Signed URL logo upload
- Brand kit saved to `growth\_clients`

### \*\*Sprint 3: Landing page rendering\*\*

- Dynamic per-client route per Section 7.1
- Testimonial slider component
- Lighthouse performance check as a sprint exit criterion

### \*\*Sprint 4: Asset generation\*\*

- `/api/og/testimonial/[id]` endpoint
- Testimonial management UI in the client dashboard
- Generated asset gallery with download

### \*\*Sprint 5: CAPI pipeline\*\*

- `lib/meta/capi.ts` per Section 9, called directly from the lead capture Server Action
- fbclid capture and cookie handling on landing pages
- Event logging and a simple admin-visible delivery status view

**\*\*Sprint 6: Handover\*\***

- Uptime monitors and backup action wired in
- Client-facing dashboard polish
- DigitalFlyer SA signs up through the real Sprint 1 flow as the first live tenant, this is the platform's own acceptance test, not a separate pilot-specific build

**## 13. Out of scope, do not build**

- Multi-seat team members per growth client beyond the single owner role.
- Google Ads integration. Mentioned as a future possibility, not part of this build.
- Bannerbear or any third-party image generation API. Rejected in favor of `@vercel/og`, already inside the paid stack.
- Autonomous ad campaign creation or budget management via the Meta Marketing API. This spec covers tracking and creative generation only, not an ad-buying agent.
- Meta's one-click CAPI Gateway as a replacement for the custom pipeline in Section 9. Worth knowing it exists as a simpler fallback for a client outside this platform, but not a component of this build.
- TikTok Events API or any ad platform beyond Meta.
- Tier-specific pilot campaign copy, ad creatives, and campaign strategy. Deferred to the real-world test case phase, after the platform in this spec is built.